

Security Audit Record: Supabase Row Level Security Exposure

South End Racquet & Health Club (southendclub.com) | Supabase project **zngbawafqjntciafhxgr**
Found, remediated and verified: **2026-08-18** | Status: **REMEDIATED -- verified in production**

This document contains no customer personal data and no API keys. Exposure is evidenced by row counts, policy definitions and HTTP status codes only. It is safe to store in the repository and to share with counsel or an insurer.

1. Summary

Two tables holding customer personal data were readable, modifiable and deletable by any member of the public. The cause was a single misconfigured Row Level Security policy on each table, granted to the Postgres `public` role instead of to `service_role`. Both policies have been replaced with correctly scoped equivalents and the closure verified against production.

Classification: broken access control (misconfigured authorisation policy). Not a credential leak. The Supabase anon key that reached these tables is embedded in public page source by design, and its disclosure is not a defect. Rotating it would not have reduced the exposure and was explicitly excluded from remediation.

Discovered incidentally on 2026-08-18 during unrelated work removing a test booking row. Deleting that row required no privileged access, which is what revealed the problem. There is no access log establishing whether the data was retrieved by a third party at any point before remediation; absence of evidence of retrieval is not evidence of absence. Whether the 231 affected individuals require notification is a disclosure decision for the owner and, if pursued, for counsel -- the club operates in California, so the CCPA applies, and the records carry marketing contact-consent flags.

2. Root cause

Row Level Security was **already enabled on all 45 tables** in the public schema. The exposure did not come from RLS being switched off. It came from the content of two policies:

tablename	polycyname	roles	cmd	qual	with_check
tour_bookings	Service role full access	{public}	ALL	true	true
tour_referrals	Service role full access on referrals	{public}	ALL	true	true

Both are named for the service role, but both are granted **TO public**. In PostgreSQL, `public` is not a synonym for the service role -- it denotes every role, including the unauthenticated anon role used by the public website. Combined with `cmd = ALL` and an unconditional `qual = true`, each policy granted anonymous users unrestricted SELECT, INSERT, UPDATE and DELETE.

The remaining 43 tables in the schema are written correctly, as `service_role_all ... {service_role}`. These two were the only tables in the database where the service-role policy was scoped to `public` -- consistent with a policy created without an explicit `TO` clause, which defaults to `public`, rather than with a deliberate grant.

The failure mode is worth recording separately from the incident. The policy was not missing and did not look wrong in a listing: it carried a reassuring name describing the intent of its author. Any review that read policy names rather than policy roles would have passed this database. Naming is not access control.

Origin. Both policies were created by migrations `001_create_tour_bookings.sql` and `002_create_tour_referrals.sql` in the `engagepro-booking-app` repository, which read:

```
-- Allow Edge Functions (service_role) full access
CREATE POLICY "Service role full access"
ON tour_bookings
```

```
FOR ALL
USING (true)
WITH CHECK (true);
```

The comment states the intent; the SQL omits the TO clause that would implement it. The next comment in the same file reads -- Block anon from reading/updating/deleting, which was never true from the moment the migration first ran. **Migration 004 now corrects both policies**, so a project rebuilt from these migrations lands in the correct state; 001 and 002 carry inline warnings and were otherwise left unmodified, migrations being history.

3. Exposure prior to remediation

Measured by live request against production using only the publicly published anon key, with a count-only probe (select=id, Prefer: count=exact, Range: 0-0) that returns totals in the response header and no row data.

Table	Rows	Personal data held	Anon access granted
public.tour_bookings	231	First/last name, email, cell phone, preferred date and time, gender, contact-consent	SELECT, INSERT, UPDATE, DELETE
public.tour_referrals	30	Name, email and phone for BOTH referrer and referee	SELECT, INSERT, UPDATE, DELETE

Individuals affected: 231 prospects, plus up to 60 further parties across the 30 referral records.

Disclosure. All of the above was retrievable by a single unauthenticated HTTP request.

Destruction. An unfiltered DELETE would have emptied either table. This was aggravated by the absence of any restore path: the Supabase project is on the Free plan, which the dashboard confirms includes neither scheduled backups nor point-in-time recovery. Separately, WordPress-side server backups exist but cannot be restored under the free migration plugin's approximately 512 MB import cap, and the hosting provider's managed backups have never been tested -- and those cover WordPress, not Supabase. The records therefore existed, and still exist, in exactly one place.

Tampering. UPDATE was open, so appointment dates and times could have been rewritten silently, producing a staff calendar disagreeing with confirmation emails already sent, with no audit trail to reconstruct the original values.

4. Correction to the original assessment methodology

The original finding inferred write exposure from HTTP status codes, reasoning that a protected table returns 401 or 403 to a DELETE regardless of filter, while an unprotected one returns 204; both returned 204. **That inference is invalid and is recorded here so it is not relied upon again.**

Under RLS, a DELETE matching no visible row does not error. It succeeds against zero rows and returns 204 -- indistinguishable from a genuinely open table. The same applies to UPDATE. A control confirmed this: central_departments grants anon exactly one policy, anon_select_departments (SELECT only), and provably has no anon DELETE or UPDATE policy -- yet returns:

```
central_departments  DELETE(no-match)=204  PATCH(no-match)=204
```

Two further corrections follow from the same cause. An INSERT rejected with HTTP 400 for a NOT NULL violation does not demonstrate that insert was permitted, because PostgreSQL evaluates column constraints before the RLS WITH CHECK clause. And a pass criterion of "every request must return 401 or 403" is wrong for SELECT: a correctly protected table returns 200 with an empty result set, which is precisely how the 34 locked-down tables in this schema behave. Applying that criterion literally would cause a successful remediation

to be misread as a failure.

The write exposure was nevertheless real -- established from the policy definitions in section 2, which grant DELETE and UPDATE outright. The original conclusion was correct; only its evidence was unsound.

5. Full schema review

The remediation plan directed that the PostgREST root document be walked to enumerate exposed tables. **That method does not work on this project** -- the root endpoint rejects the anon key with HTTP 401 and the hint that only the service-role key may be used. An earlier fallback that guessed approximately 60 plausible table names found nothing beyond the three already known, which was misleading: it missed 42 tables that exist.

Authoritative enumeration was instead obtained by privileged query (pg_class joined to pg_namespace), then confirmed empirically by probing all 45 tables with the anon key. This revealed a second application in the same database -- 43 tables prefixed central_ -- entirely absent from the original finding.

Result: 34 of 45 tables correctly deny anonymous reads. Eleven were readable. Of those, nine are intentional and appropriate; two were the exposure.

Table	Rows	Assessment
tour_bookings	231	EXPOSURE -- customer PII. Remediated (section 6).
tour_referrals	30	EXPOSURE -- PII for two parties per row. Remediated.
central_events	124	Intentional. Policy restricts to published + public.
central_module_columns	43	Reference/configuration data. Not sensitive.
central_departments	12	Non-sensitive operational data. Unconditional read.
central_option_lists	9	Configuration data. Not sensitive.
central_form_definitions	7	Form schemas. Unconditional read.
central_clubs	3	Club records. Unconditional read. See note below.
central_organizations	3	Organisation records. Unconditional read.
central_classes / central_polls / central_surveys	also 3	Intentional. Each gated on published status.

Observation for later attention, not part of this incident. The policy on central_clubs is named "Anon can read clubs by slug" but carries qual = true, so it returns every club rather than one matched by slug. The data is non-sensitive and the row count is 3, so this is not urgent. It is noted because it is the same class of defect as the root cause: a policy whose name asserts a restriction its definition does not implement.

6. Remediation applied

Each miswritten policy was dropped and replaced with an equivalent scoped to service_role, matching the pattern already used correctly by the other 43 tables:

```
drop policy "Service role full access" on public.tour_bookings;
create policy "service_role_all" on public.tour_bookings
  for all to service_role using (true) with check (true);

drop policy "Service role full access on referrals" on public.tour_referrals;
create policy "service_role_all" on public.tour_referrals
  for all to service_role using (true) with check (true);
```

One pre-existing policy was deliberately left in place: "Anon can insert bookings" on tour_bookings ({anon}, INSERT, with_check = true). It grants no USING clause and therefore no read, so

it cannot disclose or destroy data; its worst case is injection of spurious rows. Retaining it meant the remediation could not break the booking flow under any hypothesis, including the competing one considered in section 7. Closing a disclosure and destruction hole immediately, and evaluating a low-severity spam vector separately, was judged the correct order of operations. **It remains open and is listed in section 9.**

The exact rollback was prepared before the change was applied and was not needed:

```
drop policy if exists "service_role_all" on public.tour_bookings;
create policy "Service role full access" on public.tour_bookings
  for all to public using (true) with check (true);
-- and the equivalent for tour_referrals
```

7. The remediation-safety question, and a recorded disagreement

The remediation was safe only if no live widget reads or writes these tables directly. This was verified in the repository rather than assumed. A search for `rest/v1` across `live/`, `patches/` and `Website/Pages` returned no matches; a search for edge-function references returned exactly three (`book-tour`, `check-availability`, `validate-referral`). An additional check not called for by the plan searched for supabase-js SDK usage (`createClient`, `.from()`, which would construct `/rest/v1` URLs without that literal string appearing in source; no SDK is loaded anywhere in the repository. All three call sites pass the anon key as a header on a POST to an edge function, never to a table.

7.1 Recorded disagreement. During execution it was proposed that the remediation must retain an explicit INSERT grant for anon, on the reasoning that the booking form writes with that key and would otherwise fail silently. The evidence above indicated otherwise: the form posts to an edge function, which supplies its own service-role credential server-side. Granting anon broad table access to preserve the booking flow would have reinstated part of the very hole under repair.

The proposal was nonetheless partly well-founded, and is recorded rather than dismissed: an explicit "Anon can insert bookings" policy does exist, so an anon INSERT path had at some point been deliberately created. The resolution was to close the disclosure hole while leaving that policy untouched, which satisfied both positions without either having to be assumed correct. The question was then settled by measurement rather than by argument -- see section 8.

Limits of the repository evidence. The edge-function source is not held in this repository (`Components/Backend/supabase/functions/book-tour/` exists but is empty) and the repository is known to lag production. The searches above evidence the repository only. Production evidence is in section 8.

8. Verification against production

All verification was performed by command-line request. Browser verification is unreliable against this stack because of caching and was not accepted as evidence.

8.1 Anonymous access is closed. Both tables now return an empty result set and a count of zero, against a pre-change baseline of 231 and 30:

BEFORE	<code>tour_bookings</code>	HTTP/1.1 206	Content-Range: 0-0/231	
	<code>tour_referrals</code>	HTTP/1.1 206	Content-Range: 0-0/30	
AFTER	<code>tour_bookings</code>	HTTP/1.1 200	Content-Range: */0	body: []
	<code>tour_referrals</code>	HTTP/1.1 200	Content-Range: */0	body: []

Response bodies were inspected directly and are empty arrays, confirming the zero count is genuine filtering rather than a header artefact. Status codes for GET, PATCH and DELETE across the three tables are 200/204/204, which -- per section 4 -- is the correct signature of a protected table, not of an open one.

8.2 The booking flow still works, proven positively. The check-availability edge function returns "success":true with fully populated slot lists. A success response alone would have been weak evidence, because a function silently reading as anon would now see zero bookings and report every slot free -- succeeding while double-booking tours. A date sweep was therefore run to find a date with an existing booking:

```
2026-08-23    "booked_slots":["10:30 AM"]
```

CORRECTION -- this reasoning was unsound and is retracted. The argument made at the time was that the edge function had returned a booking the anon role provably could not see, and therefore must be reading the table with credentials that bypass RLS. **The premise was false.** Retrieval of the function's source on 2026-08-18 showed that check-availability never touches Supabase at all: it contains no reference to createClient, supabase or tour_bookings. Its booked_slots come from the Engage Pro CRM calendar. The response therefore said nothing about database permissions in either direction.

The conclusion survived on other evidence -- section 8.3, where book-tour demonstrably wrote to the table while anon held no access, and the function's source, which builds its client with SUPABASE_SERVICE_ROLE_KEY. Only the reasoning was wrong. It is retained here rather than deleted because it is the same failure this record criticises in section 4: an expectation about behaviour that nobody had checked against the implementation. The reviewer who produced section 4 then committed the identical error two hours later.

Operational consequence. check-availability is not a health check for database access. It will return "success":true with correct slot data even if Supabase is entirely unreachable. It must not be used to verify an RLS or database change.

8.3 The write path is verified. With owner authorisation, a probe was posted to the book-tour edge function. It returned:

```
{"success":true,"booking_id":"4a2f82a7-...", "status":"confirmed",  
  "crm_synced":true,"appointment_created":true,"supabase_available":true}
```

The function wrote to tour_bookings, synchronised the CRM and created a calendar appointment **while anon held no read or write access to the table**. This confirms the write path in production and settles section 7.1 conclusively: the booking flow requires no anonymous table access of any kind. The remaining "Anon can insert bookings" policy is therefore unnecessary as well as undesirable, and can be removed.

The probe was intended to be non-destructive and was not. It targeted a slot that check-availability reported as already booked, on the expectation that the function would reject the conflict and thereby exercise the write path without writing. The function accepted it, creating a real record, a real CRM prospect and a real staff-calendar appointment. The risk had been identified and accepted in advance; undeliverable @example.invalid test data was used, so no message reached any person. All artefacts were removed the same day: the database row by SQL (anonymous DELETE being correctly blocked by the remediation), and CRM prospect 34548 with its appointment.

The failed expectation is itself a finding. That the function accepted a double-booking is an unrelated production defect, recorded in section 9 and handed off separately.

8.4 Re-exposure during remediation

The exposure was reopened on 2026-08-18, after it had been closed. While reviewing the origin of the defect, the original CREATE POLICY statement was quoted from the migration file as evidence. It was then run against production, which recreated the policy -- again without a TO clause, and therefore again granted to public. All 231 booking records became publicly readable a second time.

It was detected within seconds by the guard built earlier the same day (npm run guard:rls, in the repository's guard chain), which probes production and fails when a table classed as denied returns rows:

```
RLS GUARD FAILED

- PII TABLE "tour_bookings" is EXPOSED -- anon read 231 rows (expected 0). HTTP 206.
  Check for a policy granted TO public or TO anon on this table.
```

The policy was dropped and closure re-verified. Exposure lasted minutes, on an endpoint no third party had reason to be watching, but it is recorded because the window is real and because of what it demonstrates.

Two lessons, both acted upon. First, the guard justified itself within hours of being written -- catching a regression that a human review would not have noticed, since the reintroduced policy looked exactly as legitimate as the original. Second, and more importantly: **the dangerous statement was copied out of a migration file that presented it as normal, working code.** Documenting the hazard in a README would not have prevented this, because the file was read directly. The broken statements in migrations 001 and 002 now carry inline warnings on the lines themselves (DO NOT RUN THE POLICY BLOCK BELOW -- SUPERSEDED BY 004), and a corrective migration 004 exists so a fresh project is never built into the exposed state.

9. Outstanding items

Item	Severity	Status
book-tour accepted double-bookings	RESOLVED	Unrelated defect, found by the section 8.3 probe: the function accepted a booking for a slot check-avail
Edge-function source location	RESOLVED	Recorded earlier in this document as unversioned. That was wrong. All three functions live in the enga
"Anon can insert bookings" policy	retained	Grants INSERT only, no read, so it cannot disclose or destroy data; it permits injection of spurious rows
No backup or restore path for the database	High	Free plan provides neither scheduled backups nor point-in-time recovery. 261 records of personal data
central_clubs policy name does not match table	Informational	Definition sensitive data, 3 rows. Same defect class as the root cause.
Whether affected individuals received notification	Open question	Not an engineering determination. No access log exists to establish whether the data was retrieved dur
Prior test booking's CRM appointment	These (e.g. 81)	Outstanding from earlier work; unrelated to this finding.

The exposure window cannot be bounded from below. The misconfiguration's creation date was not established, so the data should be treated as having been reachable for an unknown period ending 2026-08-18.

10. Actions taken and not taken

Action	Status
Repository verification of the remediation-safety assumption, incl. SDK check	DONE
Pre-change exposure baseline captured (counts only)	DONE
Backup facility checked in the Supabase dashboard	DONE -- none available (Free plan)
Manual export of the two personal-data tables	NOT DONE -- deliberate owner decision to proceed without one
Pre-change policy state captured and recorded	DONE
Full 45-table schema enumerated and probed	DONE
Two miswritten policies dropped and replaced	DONE
Post-change verification of anonymous access	DONE -- closed
Post-change verification of the booking read path	DONE -- working
End-to-end booking write path verified (book-tour)	DONE -- working; probe artefacts cleaned from both systems

Rotation of the anon key	NONE -- out of scope by design
Customer personal data read, exported or copied at any point	NONE -- counts and status codes only

The remediation was applied by the site owner in the Supabase SQL editor. Verification was performed independently by command-line request against production. No customer personal data was read, exported or copied during this work.

Source of record: handoffs/lock-down-supabase-rls.md. Regenerate with python security/generate-rls-audit-record.py. Handling rules: security/README.md.